Note: *The Achievements system has now been merged into main, and teams should have access to the AchievementsClient which is the easiest interface to interact with our API. While the feature is not as polished as we would have liked this late in the project, there should be enough in this guide for teams to begin adding achievements to their games. Please keep an eye out where limitations have been noted.*

For non-game features or teams that want to establish their own codebase to integrate with achievements, the API endpoints and AchievementsMethods are available.

Otherwise, you can follow this guide to implement what we have for Words On Stream. Please see here for explanation of the design patterns.

We currently use a helper class per game (e.g., `WordsOnStreamAchievements`) and a shared `AchievementsClient` to simplify adding achievements. Other teams can follow the same pattern instead of calling API endpoints directly, assuming their codebase has not significantly deviated from the original Muddle structure.

This guide shows how to add achievements to TicTacToe as an example. Replace TicTacToe with your own game name.

1. Add a Game Achievements Class

Create a new class in your server package, e.g., `/server/tictactoe/TicTacToeAchievements.java` .

It should:

- Store a reference to `AchievementsClient`
- Register achievements at startup
- Provide helper methods to unlock/update achievements asynchronously

```
package minigames.server.tictactoe;

import minigames.server.AchievementsClient;

public class TicTacToeAchievements {
    private final AchievementsClient client;

    public TicTacToeAchievements(AchievementsClient client) {
        this.client = client;
    }

    public void registerTicTacToeAchievements() {
        client.registerAchievement("firstWin", "TicTacToe", "First Victory", "Win your f
    }

    public void unlockFirstWin(String username) {
        client.unlockAchievement(username, "firstWin");
    }
}
```

2. Register Achievements at Server Startup

Open `/server/Main.java` and import your new class:

```
import minigames.server.tictactoe.TicTacToeAchievements;
```

Move TicTacToe setup into a dedicated helper method:

```
private void setupTicTacToe(AchievementsClient achievementsClient) {

    TicTacToeAchievements tictactoeAchievements = new TicTacToeAchievements(achievements

    // Register achievements asynchronously
    tictactoeAchievements.registerTicTacToeAchievements(success -> {
        if (success) {
            logger.info("All TicTacToe achievements registered successfully");
        } else {
            logger.warn("Failed to register some TicTacToe achievements");
        }
```

```
    });

    // Register the game server immediately
    gameRegistry.registerGameServer("TicTacToe", new TicTacToeServer(achievementsClient)
}
```

Note: Game servers are registered independent of achievement registration success.

Add your game setup in the `start()` method underneath the achievements client instantiation and pass the client in:

```
@Override
public void start(Promise<Void> promise) {
    logger.info("Our Verticle is being started by Vert.x");

    AchievementsClient achievementsClient = new AchievementsClient(vertx, port);

    // Register TicTacToeachievements and server
    setupTicTacToe(achievementsClient);

    // Start the network server
    gameServer = new MinigameNetworkServer(vertx);
    gameServer.start(port);
}
```

3. Update your Server and Game Classes

server/tictactoe/TicTacToeServer

Pass the `AchievementsClient` into your server:

```
public class TicTacToeServer implements GameServer {
    private final AchievementsClient achievementsClient;

    public TicTacToeServer(AchievementsClient achievementsClient) {
        this.achievementsClient = achievementsClient;
    }

    @Override
    public Future<RenderingPackage> newGame(String playerName) {
        TicTacToeAchievements achievements = new TicTacToeAchievements(achievementsClien
        TicTacToeGame g = new TicTacToeGame(randomName(), achievements);
        games.put(g.name, g);
        return Future.succeededFuture(g.joinGame(playerName));
    }
}
```

server/tictactoe/TicTacToeGame

***Note**: Our current code assumes that a player has entered their Player Interaction Hub `username` when joining your game, therefore the playerName will match the player's username in the database, assuming they have created an account in the Hub. This is not the most secure solution as it means any player can use any other player's username to update that username's achievements without authorising the username with a password. However, with limited time remaining for development, we wanted to find a simple workaround to demonstrate the achievements feature. For future development, we would ideally work with Coding Axolotls to make their Player Interaction Hub a global login into the Minigame Network to ensure playerName always matches a player's username in the database and would be authorised with a password.*

Store your `TicTacToeAchievements` and call it during gameplay. Use the callback to ensure the popup is only shown if unlocking succeeds:

```
public class TicTacToeGame {
    private final TicTacToeAchievements achievements;
    private final String name;

    public TicTacToeGame(String name, TicTacToeAchievements achievements) {
        this.name = name;
```

```
        this.achievements = achievements;
    }

    private void handleWin(String playerName, ArrayList<JsonObject> renderingCommands) {

    // *Note -- see note below this code block
    renderingCommands.add(new JsonObject()
                .put("command", "achievementUnlocked")
                .put("achievementId", "firstWin")
                .put("message", "Achievement unlocked: First Victory!"));

    achievements.unlockFirstWin(playerName, success -> {
        if (success) {
            renderingCommands.add(new JsonObject()
                .put("command", "achievementUnlocked")
                .put("achievementId", "firstWin")
                .put("message", "Achievement unlocked: First Victory!"));
            logger.info("Player {} unlocked achievement: firstWin", playerName);
        } else {
            logger.warn("Failed to unlock achievement for player: {}", playerName);
        }
    });
}
```

***Note:** You can see in the `handleWin` method the "achievementUnlocked" command is added to `renderingCommands` both immediately when the method runs and also when success is received in the callback of `unlockFirstWin` . We noticed that even on success, the "achievementUnlocked" command was not being sent to the client (but the logger will print the success message), and believe it is due to the asynchronous nature of the API requests and the way renderingCommands are handled in the Vertx system. Unfortunately with our group members having limited time to look into resolving the problem, our workaround is to simply send back the rendering command without calling the API so that we can simulate the success on the client. If you come up with a solution, please let us know via Slack!*

4. Handle Achievement Popups in the Client

In `/client/tictactoe/TicTacToe.java`, handle the "achievementUnlocked" command:

```
private void showAchievementPopup(String achievementId, String message) {
    // Your UI code here
}

@Override
public void execute(GameMetadata game, JsonObject command) {
    String cmd = command.getString("command");
    if ("achievementUnlocked".equals(cmd)) {
        String achievementId = command.getString("achievementId");
        String message = command.getString("message");
        if (achievementId != null && message != null) {
            showAchievementPopup(achievementId, message);
        }
    }
}
```

Comments